

Tarea 5: Tabla Hash y Árbol Binario de Búsqueda — Documento de reflexión

Igal Shturman Poplawsky — Matrícula 18139

1. Mi función hash: por qué “abc” y “cba” no colisionan

Mi función hash es la variante djb2 para cadenas (VisuAlgo, s.f.-a):

```
int HashTable<V>::Hash(std::string key)
{
    if (_capacidad <= 0) return 0;

    // Algoritmo djb2 para cadenas:
    // Multiplica por 33 y suma el caracter.
    // Preserva el orden de los caracteres (distingue "abc" y "cba").
    unsigned long hash = 5381;
    for (char currentCharacter : key)
    {
        hash = ((hash << 5) + hash) + static_cast<unsigned char>(currentCharacter);
    }

    return static_cast<int>(hash % static_cast<unsigned long>(_capacidad));
}
```

Empieza en 5381 y por cada carácter hace $\text{hash} = \text{hash} * 33 + c$ (escrito como $((\text{hash} \ll 5) + \text{hash}) + c$). Lo importante es la multiplicación en cada paso: el primer carácter se multiplica una vez por cada carácter que viene después, así que la posición pesa. Con “abc” la “a” se multiplica dos veces más; con “cba” es la “c” la que se multiplica dos veces. Por eso dan índices distintos, como exige la prueba “El hash distingue el orden de los caracteres”.

La primera versión que descarté fue la idea más natural: sumar los valores ASCII de los caracteres. Falla justo en ese caso porque la suma es conmutativa: $97 + 98 + 99$ da lo mismo sin importar el orden, así que “abc” y “cba” caían en el mismo bucket. Multiplicar por 33 en cada paso rompe esa simetría. El módulo final garantiza las otras dos propiedades: el resultado siempre cae dentro del rango de buckets y, al ser una operación determinista, la misma llave siempre da el mismo índice.

2. InOrdenIterativo frente al in-orden recursivo: ¿quién guarda los pendientes?

Mi recorrido in-orden recursivo y su auxiliar son:

```
void Tree<T>::InOrden(LinkedList<T>& resultado)
{
    InRec(_root, resultado);
}

void Tree<T>::InRec(Node* currentNode, LinkedList<T>& resultado)
{
    if (currentNode == nullptr) return;
    InRec(currentNode->left, resultado);
    resultado.Add(currentNode->data);
    InRec(currentNode->right, resultado);
}
```

Y esta es la versión iterativa, que usa mi propio `Stack<Node*>`:

```

void Tree<T>::InOrdenIterativo(LinkedList<T>& resultado)
{
    // In-orden sin recursion: nodeStack guarda los nodos pendientes.
    // Es el mismo trabajo que hacia la pila de llamadas del compilador.
    Stack<Node*> nodeStack;
    Node* currentNode = _root;

    while (currentNode != nullptr || !nodeStack.IsEmpty())
    {
        // Bajar lo mas a la izquierda posible, apilando el camino.
        while (currentNode != nullptr)
        {
            nodeStack.Push(currentNode);
            currentNode = currentNode->left;
        }
        // Ya no se puede bajar: sacar uno, procesarlo, ir a su derecha.
        currentNode = nodeStack.Pop();
        resultado.Add(currentNode->data);
        currentNode = currentNode->right;
    }
}

```

En la versión iterativa el nodeStack guarda los nodos pendientes: los ancestros por los que ya bajé hacia la izquierda y cuyo subárbol izquierdo todavía se está explorando. El ciclo baja todo lo posible apilando el camino; cuando ya no puede bajar, saca un nodo de la pila (ese ya tiene su izquierda terminada), lo agrega al resultado y se mueve a su hijo derecho. La pila nunca contiene nodos al azar: siempre es el camino de la raíz al nodo actual, es decir, exactamente los nodos que están “en espera”.

En la versión recursiva eso mismo lo guardaba la pila de llamadas del compilador, sin que yo la viera: cada llamada a InRec deja un registro de activación con el currentNode de ese nivel y el punto donde debe continuar al regresar. Bajar a la izquierda es apilar un registro; regresar del hijo izquierdo es desapilarlo y seguir con la línea del nodo actual. Comparadas lado a lado, hacen lo mismo: la única diferencia es quién administra la pila, el compilador o yo (LearnCpp.com, s.f.).

3. Las dos versiones de BFS: ¿cuál es más lenta y por qué?

Mi BFS con cola (usa mi LinkedQueue<Node*>) es:

```

void Tree<T>::PorNiveles(LinkedList<T>& resultado)
{
    // BFS con cola: mismo esquema que el DFS iterativo,
    // pero la cola produce orden por niveles.
    if (_root == nullptr) return;

    LinkedQueue<Node*> nodeQueue;
    nodeQueue.Enqueue(_root);

    while (!nodeQueue.IsEmpty())
    {
        Node* currentNode = nodeQueue.Dequeue();
        resultado.Add(currentNode->data);
        if (currentNode->left != nullptr) nodeQueue.Enqueue(currentNode->left);
        if (currentNode->right != nullptr) nodeQueue.Enqueue(currentNode->right);
    }
}

```

Y la versión recursiva, sin cola, con su auxiliar por nivel:

```

void Tree<T>::PorNivelesRecursivo(LinkedList<T>& resultado)
{

```

```

// Sin cola: se visita cada nivel por separado.
// Mas lento porque los nodos de arriba se recorren
// una vez por cada nivel que hay debajo de ellos.
int treeHeight = GetAltura();
for (int nivel = 1; nivel <= treeHeight; nivel++)
{
    NivelRec(_root, nivel, resultado);
}
}

void Tree<T>::NivelRec(Node* currentNode, int nivel, LinkedList<T>& resultado)
{
    if (currentNode == nullptr) return;
    if (nivel == 1)
    {
        resultado.Add(currentNode->data);
        return;
    }
    NivelRec(currentNode->left, nivel - 1, resultado);
    NivelRec(currentNode->right, nivel - 1, resultado);
}
}

```

La versión recursiva es la más lenta. Razón: para el nivel k vuelve a bajar desde la raíz, así que la raíz se visita tantas veces como niveles tenga el árbol (h veces), sus hijos $h - 1$ veces, y así sucesivamente. En total el trabajo es proporcional a $n \cdot h$, mientras que la versión con cola visita cada nodo exactamente una vez porque cada nodo se encola y se saca una sola vez: $O(n)$. Es el mismo resultado con distinto costo, y muestra que DFS y BFS solo se diferencian en dónde guardan los pendientes, pila o cola (VisuAlgo, s.f.-c).

La diferencia se nota más cuando el árbol es alto: en un árbol degenerado (insertar en orden, altura igual al tamaño n) la versión recursiva hace del orden de n^2 visitas contra n de la versión con cola. En un árbol bajo y balanceado apenas se percibe, porque h es pequeña.

Referencias

Cplusplus.com. (s.f.). C++ reference. <https://cplusplus.com/reference/>

LearnCpp.com. (s.f.). Learn C++. <https://www.learncpp.com/>

VisuAlgo. (s.f.-a). Hash table. <https://visualgo.net/es/hashtable>

VisuAlgo. (s.f.-b). Binary search tree. <https://visualgo.net/es/bst>

VisuAlgo. (s.f.-c). Graph traversal: DFS/BFS. <https://visualgo.net/es/dfsdfs>